

Operating Systems Lab Assignment 7: Synchronization and Scheduling

Christian Clarke

March 20, 2026

1 Introduction

This report covers the synchronization and scheduling lab using POSIX threads, mutexes, and condition variables.

2 Exercise 1: Hello World

```
#include <stdio.h>
#include <pthread.h>

pthread_mutex_t lock;
pthread_cond_t cv;
int hello = 0;

void* print_hello(void* arg) {
    pthread_mutex_lock(&lock);
    hello += 1;
    printf("First_line_(hello=%d)\n", hello);
    pthread_cond_signal(&cv);
    pthread_mutex_unlock(&lock);
    return NULL;
}

int main() {
    pthread_t thread;

    pthread_mutex_init(&lock, NULL);
    pthread_cond_init(&cv, NULL);

    pthread_create(&thread, NULL, print_hello, NULL);

    pthread_mutex_lock(&lock);
    while (hello < 1) {
        pthread_cond_wait(&cv, &lock);
    }
    printf("Second_line_(hello=%d)\n", hello);
    pthread_mutex_unlock(&lock);

    pthread_join(thread, NULL);

    pthread_mutex_destroy(&lock);
    pthread_cond_destroy(&cv);
    return 0;
}
```

```
}
```

Explanation: The child thread increments the shared variable and signals the condition variable. The main thread waits until the shared value becomes 1, which prevents a race condition and guarantees the output order.

Analysis: Without synchronization, the second line could print before the first line or the shared variable could be read too early.

Screenshot: Insert your compilation and execution screenshot here.

3 Exercise 2: SpaceX Problems

```
#include <stdio.h>
#include <pthread.h>

pthread_mutex_t lock = PTHREAD_MUTEX_INITIALIZER;
pthread_cond_t cv = PTHREAD_COND_INITIALIZER;
int n = 3;

void* counter(void* arg) {
    pthread_mutex_lock(&lock);
    while (n > 0) {
        printf("%d\n", n);
        n--;
        if (n == 0) {
            pthread_cond_signal(&cv);
        }
    }
    pthread_mutex_unlock(&lock);
    return NULL;
}

void* announcer(void* arg) {
    pthread_mutex_lock(&lock);
    while (n != 0) {
        pthread_cond_wait(&cv, &lock);
    }
    printf("FALCON_HEAVY_TOUCHDOWN!\n");
    pthread_mutex_unlock(&lock);
    return NULL;
}

int main() {
    pthread_t t1, t2;

    pthread_create(&t1, NULL, counter, NULL);
    pthread_create(&t2, NULL, announcer, NULL);

    pthread_join(t1, NULL);
    pthread_join(t2, NULL);

    pthread_mutex_destroy(&lock);
    pthread_cond_destroy(&cv);
    return 0;
}
```

Explanation: The counter thread holds the mutex while counting down and signals the announcer when the countdown reaches 0. The announcer waits on the condition variable until that happens.

Analysis: This ensures the touchdown message prints only after 3, 2, 1.

Screenshot: Insert your compilation and execution screenshot here.

4 Exercise 3: I Love You, Unconditionally!

```
#include <stdio.h>
#include <pthread.h>

pthread_mutex_t lock;
pthread_cond_t cv;
int subaru = 0;

void* helper(void* arg) {
    pthread_mutex_lock(&lock);
    subaru += 1;
    pthread_cond_signal(&cv);
    pthread_mutex_unlock(&lock);
    return NULL;
}

int main() {
    pthread_t thread;

    pthread_mutex_init(&lock, NULL);
    pthread_cond_init(&cv, NULL);

    pthread_create(&thread, NULL, helper, NULL);

    pthread_mutex_lock(&lock);
    while (subaru != 1) {
        pthread_cond_wait(&cv, &lock);
    }

    if (subaru == 1) {
        printf("I love Emilia!\n");
    } else {
        printf("I love Rem!\n");
    }
    pthread_mutex_unlock(&lock);

    pthread_join(thread, NULL);

    pthread_mutex_destroy(&lock);
    pthread_cond_destroy(&cv);
    return 0;
}
```

Explanation: The helper thread changes the shared variable and signals the main thread. The main thread waits until the value becomes 1, guaranteeing the correct message.

Analysis: The condition variable prevents the main thread from checking the shared value too early.

Screenshot: Insert your compilation and execution screenshot here.

5 Exercise 4: Locking Up the Floopies

```
#include <stdio.h>
#include <pthread.h>
```

```

typedef struct account_t {
    pthread_mutex_t lock;
    int balance;
    long uuid;
} account_t;

typedef struct transfer_args_t {
    account_t* donor;
    account_t* recipient;
    float amount;
} transfer_args_t;

void transfer(account_t *donor, account_t *recipient, float amount) {
    account_t *first = (donor->uuid < recipient->uuid) ? donor : recipient;
    account_t *second = (donor->uuid < recipient->uuid) ? recipient : donor;

    pthread_mutex_lock(&first->lock);
    pthread_mutex_lock(&second->lock);

    if (donor->balance < amount) {
        printf("Insufficient funds.\n");
    } else {
        donor->balance -= (int)amount;
        recipient->balance += (int)amount;
        printf("Transferred %.0f from account %ld to %ld\n",
            amount, donor->uuid, recipient->uuid);
    }

    pthread_mutex_unlock(&second->lock);
    pthread_mutex_unlock(&first->lock);
}

void* transfer_thread(void* arg) {
    transfer_args_t* args = (transfer_args_t*)arg;
    transfer(args->donor, args->recipient, args->amount);
    return NULL;
}

int main() {
    account_t acc1 = {PTHREAD_MUTEX_INITIALIZER, 1000, 1};
    account_t acc2 = {PTHREAD_MUTEX_INITIALIZER, 500, 2};

    pthread_t t1, t2;

    transfer_args_t a1 = {&acc1, &acc2, 200};
    transfer_args_t a2 = {&acc2, &acc1, 100};

    pthread_create(&t1, NULL, transfer_thread, &a1);
    pthread_create(&t2, NULL, transfer_thread, &a2);

    pthread_join(t1, NULL);
    pthread_join(t2, NULL);

    printf("Final balance acc1: %d\n", acc1.balance);
    printf("Final balance acc2: %d\n", acc2.balance);

    pthread_mutex_destroy(&acc1.lock);
    pthread_mutex_destroy(&acc2.lock);
}

```

```
    return 0;
}
```

Explanation: Transfers lock both accounts in a consistent order based on account ID. This avoids circular wait and prevents deadlock.

Analysis: If two threads lock the two accounts in opposite orders, each can block forever waiting on the other lock.

Screenshot: Insert your compilation and execution screenshot here.

6 Exercise 5: Baking with Condition Variables

```
#include <stdio.h>
#include <pthread.h>
#include <stdbool.h>

int numBatterInBowl = 0;
int numEggInBowl = 0;
bool readyToEat = false;
int cakesMade = 0;
const int MAX_CAKES = 3;

pthread_mutex_t lock;
pthread_cond_t needIngredients, readyToBake, startEating;

void addBatter() {
    numBatterInBowl += 1;
    printf("Added␣batter.␣Batter=%d␣Eggs=%d\n", numBatterInBowl, numEggInBowl);
}

void addEgg() {
    numEggInBowl += 1;
    printf("Added␣egg.␣Batter=%d␣Eggs=%d\n", numBatterInBowl, numEggInBowl);
}

void heatBowl() {
    readyToEat = true;
    numBatterInBowl = 0;
    numEggInBowl = 0;
    printf("Cake␣baked!\n");
}

void eatCake() {
    readyToEat = false;
    cakesMade++;
    printf("Cake␣eaten!␣Total␣cakes=%d\n", cakesMade);
}

void* batterAdder(void* arg) {
    while (1) {
        pthread_mutex_lock(&lock);

        if (cakesMade >= MAX_CAKES) {
            pthread_cond_broadcast(&readyToBake);
            pthread_cond_broadcast(&startEating);
            pthread_cond_broadcast(&needIngredients);
            pthread_mutex_unlock(&lock);
            break;
        }
    }
}
```

```

    }

    while ((numBatterInBowl >= 1 || readyToEat) && cakesMade < MAX_CAKES) {
        pthread_cond_wait(&needIngredients, &lock);
    }

    if (cakesMade < MAX_CAKES && !readyToEat && numBatterInBowl < 1) {
        addBatter();
        pthread_cond_signal(&readyToBake);
    }

    pthread_mutex_unlock(&lock);
}
return NULL;
}

void* eggBreaker(void* arg) {
    while (1) {
        pthread_mutex_lock(&lock);

        if (cakesMade >= MAX_CAKES) {
            pthread_cond_broadcast(&readyToBake);
            pthread_cond_broadcast(&startEating);
            pthread_cond_broadcast(&needIngredients);
            pthread_mutex_unlock(&lock);
            break;
        }

        while ((numEggInBowl >= 2 || readyToEat) && cakesMade < MAX_CAKES) {
            pthread_cond_wait(&needIngredients, &lock);
        }

        if (cakesMade < MAX_CAKES && !readyToEat && numEggInBowl < 2) {
            addEgg();
            pthread_cond_signal(&readyToBake);
        }

        pthread_mutex_unlock(&lock);
    }
    return NULL;
}

void* bowlHeater(void* arg) {
    while (1) {
        pthread_mutex_lock(&lock);

        if (cakesMade >= MAX_CAKES) {
            pthread_cond_broadcast(&startEating);
            pthread_mutex_unlock(&lock);
            break;
        }

        while ((numBatterInBowl < 1 || numEggInBowl < 2 || readyToEat) &&
            cakesMade < MAX_CAKES) {
            pthread_cond_wait(&readyToBake, &lock);
        }

        if (cakesMade < MAX_CAKES && numBatterInBowl >= 1 && numEggInBowl >= 2 &&
            !readyToEat) {

```

```

        heatBowl();
        pthread_cond_signal(&startEating);
    }

    pthread_mutex_unlock(&lock);
}
return NULL;
}

void* cakeEater(void* arg) {
    while (1) {
        pthread_mutex_lock(&lock);

        if (cakesMade >= MAX_CAKES) {
            pthread_cond_broadcast(&needIngredients);
            pthread_mutex_unlock(&lock);
            break;
        }

        while (!readyToEat && cakesMade < MAX_CAKES) {
            pthread_cond_wait(&startEating, &lock);
        }

        if (cakesMade < MAX_CAKES && readyToEat) {
            eatCake();
            pthread_cond_broadcast(&needIngredients);
        }

        pthread_mutex_unlock(&lock);
    }
    return NULL;
}

int main() {
    pthread_t batter, egg1, egg2, heater, eater;

    pthread_mutex_init(&lock, NULL);
    pthread_cond_init(&needIngredients, NULL);
    pthread_cond_init(&readyToBake, NULL);
    pthread_cond_init(&startEating, NULL);

    pthread_create(&batter, NULL, batterAdder, NULL);
    pthread_create(&egg1, NULL, eggBreaker, NULL);
    pthread_create(&egg2, NULL, eggBreaker, NULL);
    pthread_create(&heater, NULL, bowlHeater, NULL);
    pthread_create(&eater, NULL, cakeEater, NULL);

    pthread_join(batter, NULL);
    pthread_join(egg1, NULL);
    pthread_join(egg2, NULL);
    pthread_join(heater, NULL);
    pthread_join(eater, NULL);

    pthread_mutex_destroy(&lock);
    pthread_cond_destroy(&needIngredients);
    pthread_cond_destroy(&readyToBake);
    pthread_cond_destroy(&startEating);

    return 0;
}

```

```
}
```

Explanation: Separate threads add batter, add eggs, heat the bowl, and eat cake. Condition variables force the proper sequence: ingredients first, then baking, then eating.

Analysis: This avoids invalid states like baking before the ingredients are ready.

Screenshot: Insert your compilation and execution screenshot here.

7 Exercise 6: Priority Donation in Transfer

```
#include <stdio.h>
#include <pthread.h>

typedef struct account_t {
    pthread_mutex_t lock;
    int balance;
    long uuid;
    int priority;
} account_t;

typedef struct transfer_args_t {
    account_t* donor;
    account_t* recipient;
    float amount;
    int priority;
} transfer_args_t;

void transfer(account_t *donor, account_t *recipient, float amount, int
thread_priority) {
    account_t *first = (donor->uuid < recipient->uuid) ? donor : recipient;
    account_t *second = (donor->uuid < recipient->uuid) ? recipient : donor;

    pthread_mutex_lock(&first->lock);
    if (first->priority < thread_priority) first->priority = thread_priority;

    pthread_mutex_lock(&second->lock);
    if (second->priority < thread_priority) second->priority = thread_priority;

    if (donor->balance < amount) {
        printf("[priority%d] Insufficient funds.\n", thread_priority);
    } else {
        donor->balance -= (int)amount;
        recipient->balance += (int)amount;
        printf("[priority%d] Transferred%.0f from account %ld to %ld\n",
            thread_priority, amount, donor->uuid, recipient->uuid);
    }

    second->priority = 0;
    first->priority = 0;

    pthread_mutex_unlock(&second->lock);
    pthread_mutex_unlock(&first->lock);
}

void* transfer_thread(void* arg) {
    transfer_args_t* params = (transfer_args_t*)arg;
    transfer(params->donor, params->recipient, params->amount, params->priority);
    return NULL;
}
```



```

}

int main() {
    account_t acc1 = {PTHREAD_MUTEX_INITIALIZER, 1000, 1, 0};
    account_t acc2 = {PTHREAD_MUTEX_INITIALIZER, 500, 2, 0};

    pthread_t t1, t2;

    transfer_args_t p1 = {&acc1, &acc2, 200, 2};
    transfer_args_t p2 = {&acc2, &acc1, 100, 1};

    pthread_create(&t1, NULL, transfer_thread, &p1);
    pthread_create(&t2, NULL, transfer_thread, &p2);

    pthread_join(t1, NULL);
    pthread_join(t2, NULL);

    printf("Final balance acc1: %d\n", acc1.balance);
    printf("Final balance acc2: %d\n", acc2.balance);

    pthread_mutex_destroy(&acc1.lock);
    pthread_mutex_destroy(&acc2.lock);
    return 0;
}

```

Explanation: The code simulates priority donation by temporarily raising the priority field of locked accounts when a higher-priority transfer thread waits.

Analysis: Priority donation helps explain how priority inversion can be reduced when a low-priority holder blocks a higher-priority thread.

Screenshot: Insert your compilation and execution screenshot here.

8 Exercise 7: Barrier Synchronization

```

#include <stdio.h>
#include <pthread.h>

#define NUM_THREADS 4

pthread_mutex_t lock;
pthread_cond_t cv;
int count = 0;

void barrier_point() {
    pthread_mutex_lock(&lock);
    count++;

    if (count < NUM_THREADS) {
        while (count < NUM_THREADS) {
            pthread_cond_wait(&cv, &lock);
        }
    } else {
        pthread_cond_broadcast(&cv);
    }

    pthread_mutex_unlock(&lock);
}

```

```

void* worker(void* arg) {
    int id = *(int*)arg;
    printf("Thread%d: Before barrier\n", id);
    barrier_point();
    printf("Thread%d: After barrier\n", id);
    return NULL;
}

int main() {
    pthread_t threads[NUM_THREADS];
    int ids[NUM_THREADS];

    pthread_mutex_init(&lock, NULL);
    pthread_cond_init(&cv, NULL);

    for (int i = 0; i < NUM_THREADS; i++) {
        ids[i] = i;
        pthread_create(&threads[i], NULL, worker, &ids[i]);
    }

    for (int i = 0; i < NUM_THREADS; i++) {
        pthread_join(threads[i], NULL);
    }

    pthread_mutex_destroy(&lock);
    pthread_cond_destroy(&cv);
    return 0;
}

```

Explanation: Each thread reaches the barrier and waits until all threads arrive. The last thread broadcasts a wakeup so all threads can continue.

Analysis: This guarantees that no thread prints “After barrier” before every thread has reached the barrier.

Screenshot: Insert your compilation and execution screenshot here.

9 Exercise 8: Readers-Writers with Priority

```

#include <stdio.h>
#include <pthread.h>

pthread_mutex_t lock;
pthread_cond_t reader_cv, writer_cv;
int reader_count = 0, writer_waiting = 0, writer_active = 0;
int shared_data = 0;

void* reader(void* arg) {
    int id = *(int*)arg;

    pthread_mutex_lock(&lock);
    while (writer_waiting > 0 || writer_active) {
        pthread_cond_wait(&reader_cv, &lock);
    }
    reader_count++;
    pthread_mutex_unlock(&lock);

    printf("Reader%d reads: %d\n", id, shared_data);
}

```

```

    pthread_mutex_lock(&lock);
    reader_count--;
    if (reader_count == 0) {
        pthread_cond_signal(&writer_cv);
    }
    pthread_mutex_unlock(&lock);

    return NULL;
}

void* writer(void* arg) {
    int id = *(int*)arg;

    pthread_mutex_lock(&lock);
    writer_waiting++;
    while (reader_count > 0 || writer_active) {
        pthread_cond_wait(&writer_cv, &lock);
    }
    writer_waiting--;
    writer_active = 1;

    shared_data++;
    printf("Writer %d writes: %d\n", id, shared_data);

    writer_active = 0;
    if (writer_waiting > 0) {
        pthread_cond_signal(&writer_cv);
    } else {
        pthread_cond_broadcast(&reader_cv);
    }
    pthread_mutex_unlock(&lock);

    return NULL;
}

int main() {
    pthread_t readers[3], writers[2];
    int rids[3] = {1, 2, 3};
    int wids[2] = {1, 2};

    pthread_mutex_init(&lock, NULL);
    pthread_cond_init(&reader_cv, NULL);
    pthread_cond_init(&writer_cv, NULL);

    for (int i = 0; i < 3; i++) {
        pthread_create(&readers[i], NULL, reader, &rids[i]);
    }
    for (int i = 0; i < 2; i++) {
        pthread_create(&writers[i], NULL, writer, &wids[i]);
    }

    for (int i = 0; i < 3; i++) {
        pthread_join(readers[i], NULL);
    }
    for (int i = 0; i < 2; i++) {
        pthread_join(writers[i], NULL);
    }

    pthread_mutex_destroy(&lock);

```

```

    pthread_cond_destroy(&reader_cv);
    pthread_cond_destroy(&writer_cv);
    return 0;
}

```

Explanation: Readers can run together, but they wait if a writer is active or waiting. Writers wait until there are no readers.

Analysis: Writer priority is enforced by blocking new readers when writers are waiting.

Screenshot: Insert your compilation and execution screenshot here.

10 Exercise 9: Thread Pool

```

#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <unistd.h>

#define NUM_THREADS 4

typedef struct {
    void (*task)(int);
    int arg;
} Task;

typedef struct {
    Task* queue;
    int head, tail, count, size;
    pthread_mutex_t lock;
    pthread_cond_t not_empty, not_full;
} ThreadSafeQueue;

void queue_init(ThreadSafeQueue* q, int size) {
    q->queue = malloc(sizeof(Task) * size);
    q->head = q->tail = q->count = 0;
    q->size = size;
    pthread_mutex_init(&q->lock, NULL);
    pthread_cond_init(&q->not_empty, NULL);
    pthread_cond_init(&q->not_full, NULL);
}

void queue_push(ThreadSafeQueue* q, Task task) {
    pthread_mutex_lock(&q->lock);
    while (q->count == q->size) {
        pthread_cond_wait(&q->not_full, &q->lock);
    }
    q->queue[q->tail] = task;
    q->tail = (q->tail + 1) % q->size;
    q->count++;
    pthread_cond_signal(&q->not_empty);
    pthread_mutex_unlock(&q->lock);
}

Task queue_pop(ThreadSafeQueue* q) {
    pthread_mutex_lock(&q->lock);
    while (q->count == 0) {
        pthread_cond_wait(&q->not_empty, &q->lock);
    }
}

```

```

    Task task = q->queue[q->head];
    q->head = (q->head + 1) % q->size;
    q->count--;
    pthread_cond_signal(&q->not_full);
    pthread_mutex_unlock(&q->lock);
    return task;
}

void queue_destroy(ThreadSafeQueue* q) {
    free(q->queue);
    pthread_mutex_destroy(&q->lock);
    pthread_cond_destroy(&q->not_empty);
    pthread_cond_destroy(&q->not_full);
}

void sample_task(int arg) {
    printf("Task executed with arg: %d\n", arg);
}

void* worker(void* arg) {
    ThreadSafeQueue* q = (ThreadSafeQueue*)arg;

    while (1) {
        Task task = queue_pop(q);

        if (task.task == NULL) {
            break;
        }

        task.task(task.arg);
    }

    return NULL;
}

int main() {
    ThreadSafeQueue q;
    queue_init(&q, 10);

    pthread_t threads[NUM_THREADS];

    for (int i = 0; i < NUM_THREADS; i++) {
        pthread_create(&threads[i], NULL, worker, &q);
    }

    for (int i = 0; i < 10; i++) {
        Task t;
        t.task = sample_task;
        t.arg = i;
        queue_push(&q, t);
    }

    for (int i = 0; i < NUM_THREADS; i++) {
        Task stop_task;
        stop_task.task = NULL;
        stop_task.arg = 0;
        queue_push(&q, stop_task);
    }
}

```

```
    for (int i = 0; i < NUM_THREADS; i++) {  
        pthread_join(threads[i], NULL);  
    }  
  
    queue_destroy(&q);  
    return 0;  
}
```

Explanation: A fixed set of worker threads repeatedly pulls tasks from a shared queue instead of creating a new thread for every task.

Analysis: This improves efficiency by reusing worker threads and protecting the queue with a mutex and condition variables.

Screenshot: Insert your compilation and execution screenshot here.